

8-5 · 데이터 파이프라인 구축 및 최종 RAG 챗봇 개발 · 이론 1H

# 최종 아키텍처 점검

8주를 한 장으로, 배치와 실시간이 만나는 곳

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 1시간 뒤에는

---

- ① 8주 전 구간을 하나의 아키텍처 그림으로 그려 데이터 흐름을 설명한다
- ② 배치(DAG)와 실시간(챗봇)이 Qdrant 컬렉션 하나로 만나는 구조를 말한다
- ③ 8-3 버전에서 최종본으로 진화하는 다섯 가지를 안다
- ④ 전 구간을 관통하는 엔드투엔드 검증이 왜 강한지 설명한다

오늘은 새 기술을 배우지 않습니다 — 8주의 조각을 하나의 시스템으로 묶는 날

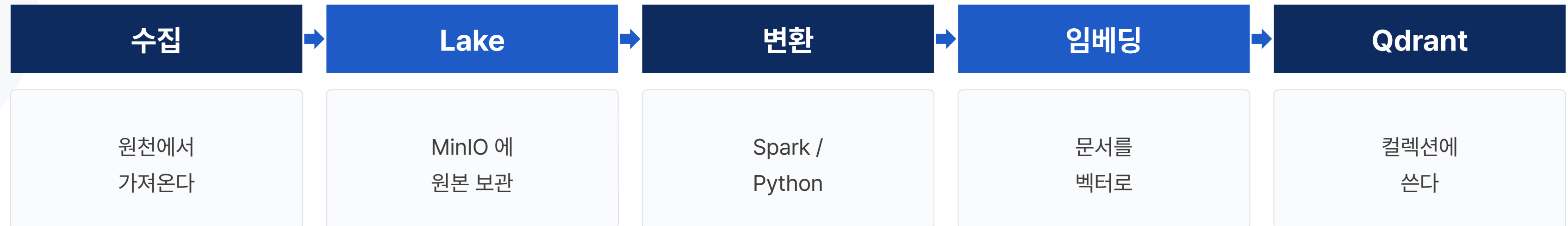
# 여덟 주 — 배운 것과 최종 시스템에서의 자리

| 주차               | 무엇을 배웠나      | 최종 시스템에서의 자리      |
|------------------|--------------|-------------------|
| 1주 Linux·Docker  | 컨테이너·Compose | 전 구성 요소가 컨테이너로 뜬다 |
| 2주 DB·수집         | SQL·크롤링·ERD  | 원천에서 데이터를 가져온다    |
| 3주 Pandas·DuckDB | 데이터 다루기      | 변환 로직의 기초         |
| 4주 Lake·Iceberg  | MinIO·테이블 형식 | 원본 보관(Lake)       |
| 5주 Spark         | 분산 처리        | 대용량 변환            |
| 6주 Kafka·CDC     | 스트리밍         | 실시간 수집 경로         |
| 7주 LLM·RAG       | 임베딩·검색·생성    | VectorDB 와 챗봇     |
| 8주 Airflow       | 오케스트레이션      | 전부를 자동으로 잇는다      |

1주의 컨테이너 위에 2~7주의 조각이 올라가고, 8주 Airflow 가 전부를 자동으로 잇는다

## 2. 전체 아키텍처

# 배치 흐름 — Airflow 가 매일 자동으로



실시간 흐름은 반대편 — 질문 → 임베딩 → Qdrant 검색 → LLM → 답변 + 출처

두 흐름은 서로를 모른다 — 만나는 곳은 Qdrant 의 컬렉션 하나뿐, DAG 는 쓰고 챗봇은 읽는다

## 2. 전체 아키텍처

# 주방(배치)과 홀(실시간) — 같은 진열대를 쓴다

## 배치 — Airflow 가 매일

원천 → 수집 → Lake → 변환 → 임베딩 → Qdrant

- DAG 가 컬렉션에 쓴다
- 주방이 멈춰도 진열대에 있는 것은 남는다
- DAG 가 실패해도 챗봇은 계속 답한다(내용은 낡는다)

VS

## 실시간 — 사람이 쓸 때마다

질문 → 임베딩 → Qdrant 검색 → LLM → 답변 + 출처

- 챗봇이 컬렉션에서 읽는다
- 한쪽을 고쳐도 다른 쪽을 멈추지 않는다
- 「얼마나 낡았나」는 payload 의 적재 시각으로 안다

# 8-3 버전에서 최종본으로 — 다섯 가지

| 항목     | 8-3 버전        | 최종본                  |
|--------|---------------|----------------------|
| 수집     | DOCS 상수 고정    | incoming/ 폴더의 파일을 읽음 |
| 스케줄    | None (수동만)    | @daily               |
| 재시도    | 없음            | retries=2, 3분 간격     |
| 포인트 id | 순번(enumerate) | 내용 기반 uuid5          |
| 빈 입력   | 오류            | 건너뛰고 정상 종료           |

순번 id 는 문서 순서가 바뀌면 엉뚱한 자리를 덮어쓴다 — 내용 기반 uuid5 는 몇 번을 돌려도 한 건

### 3. 무엇이 진화하는가

## 역등 id — 경로로도 내용으로도 만들 수 있다

- 1 순번(enumerate)이면**  
문서 순서가 바뀔 때 엉뚱한 자리를 덮어쓴다
- 2 내용 기반 uuid5 면**  
같은 문서는 항상 같은 id — 몇 번을 돌려도 한 건
- 3 7-4 에서 이미 했던 것**  
경로로 만들던 id 를 내용으로 만드는 것뿐

코드 한 줄

**uuid5**  
**내용 기반**

#### 4. 엔드투엔드 검증

# 검사 하나로 일곱 구간을 관통한다

- 1 질문 하나**  
「새 문서가 답변에 나오는가」 — 어디가 끊겨도 실패하므로 가장 강한 검사
- 2 통과하면**  
수집·저장·변환·임베딩·적재·검색·생성 일곱 구간이 전부 살아 있다는 뜻
- 3 실패하면**  
Grid 와 로그로 어디서 끊겼는지 좁혀 간다 — 태스크를 나눠 둔 것이 여기서 값을 한다
- 4 배포할 때마다**  
이 검증을 돌린다 — 사람이 매번 눈으로 확인하는 것보다 확실하다

# 이 1시간에서 꼭 가져갈 것

---

- 1 배치와 실시간은 저장소 하나를 사이에 두고 만난다
- 2 두 흐름이 서로를 모르게 두는 것이 좋은 설계다
- 3 역등 id 는 경로로도 내용으로도 만들 수 있다
- 4 전 구간을 관통하는 검증 하나가 단계별 확인보다 강하다
- 5 자동화의 목적은 「사람이 안 봐도 돌게」 만드는 것이다

8-5 · 이론 1H 마무리

## 다음은 통합 구축 실습 (1.5H)

최종 DAG 를 완성하고, 챗봇에 출처·갱신 시각·지식베이스 상태를 붙여 두 흐름이 만나는 지점을 화면으로 확인합니다

✓ 전 구성 요소 전수 점검

✓ rag\_pipeline\_final — @daily·재시도·uuid5

✓ 근거에 적재 시각, 사이드바에 문서 수

✓ 설정을 한곳에, 컬렉션 이름 세 곳 대조

준비할 것

Airflow·Qdrant 기동 · 8-4 가상환경과 키